

MLOps

Machine Learning Pipelines: Introduction – Data – Model Development

Session 3

Juan Ramírez
jramirezg@faculty.ie.edu

What is a Machine Learning model?

In Theory

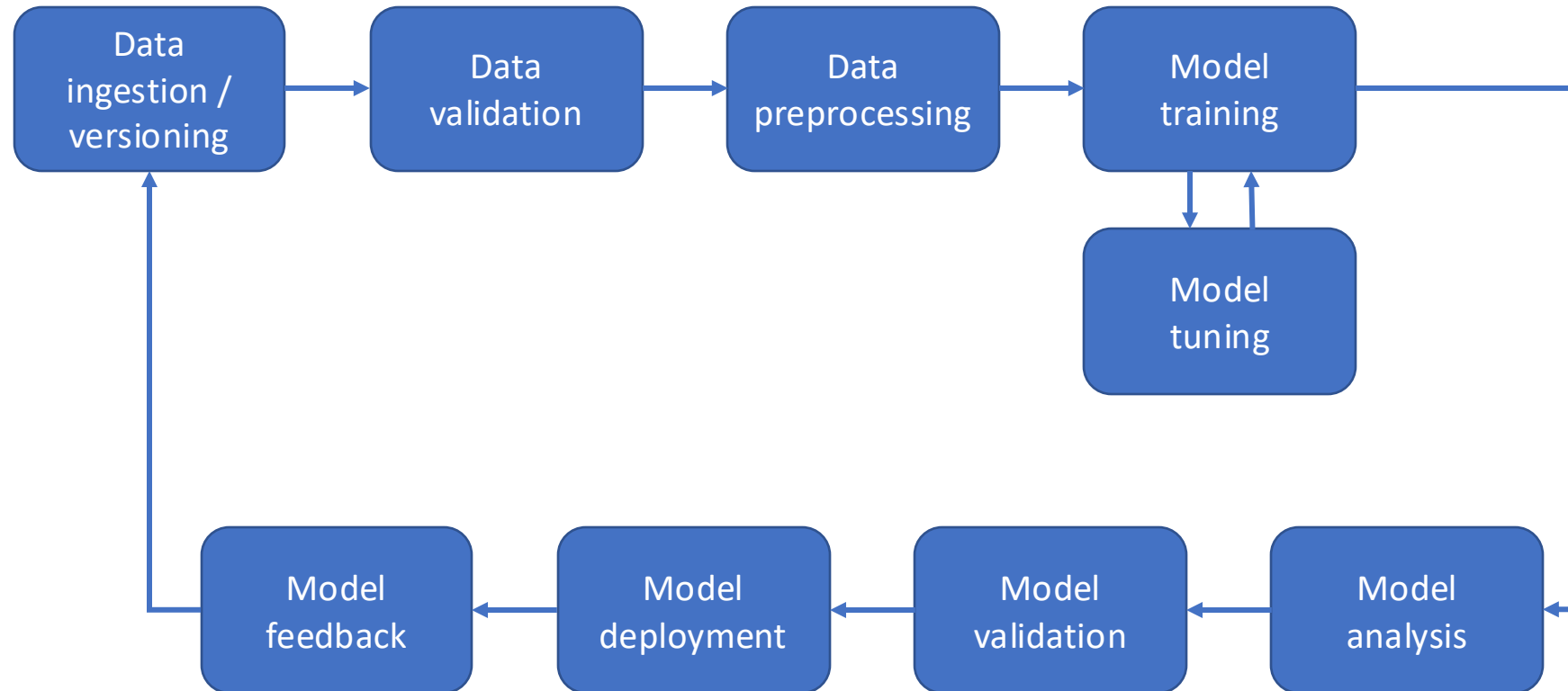
- A machine learning model is a projection of reality; that is, **it's a partial and approximate representation** of some aspect (or aspects) of a real thing or process.
- Which aspects are represented often depends on what is available and useful.
- A machine learning model, once trained, is a mathematical formula that **generates a result when fed some inputs**—for example a probability estimation of some event happening or a classification in a category.



In Practice

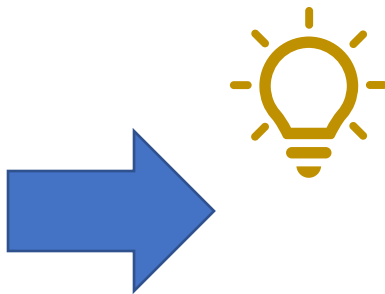
- A model is the **set of parameters necessary to rebuild and apply the formula**. It is usually stateless and deterministic (i.e., the same inputs always produce the same outputs).
- This includes also the **transformations** needed to from the input data
- And the **inference**: a value plus the possible derived data (like a classification or a decision).
- Given this description in practice, it usually does not make a difference whether the model is ML-based or not: it is just a **computable mathematical function applied to the input data**, one row at a time.

Machine Learning Pipeline



Why Machine Learning Pipelines

- Automation of the model life cycle steps
- When new training data becomes available, a workflow which includes data validation, preprocessing, model training, analysis and deployment **should be triggered**
- Benefits:
 - Ability to focus on new models, not maintaining existing models
 - Prevention of bugs (for example: one common source of bugs is a change in the preprocessing step after a model was trained -> very difficult to debug)
 - Experiment tracking and model release management
 - Standardization



- More development time for novel models
- Simpler processes to update existing models
- Less time spent to reproduce models

Pipeline Orchestration

- All the components of a machine learning pipeline need to be orchestrated, so that they are executed in the correct order
- Inputs to a component must be computed before a component is executed
- Some tools:
 - Apache Beam
 - Kubeflow Pipelines for Kubernetes infrastructure
 - Apache Airflow

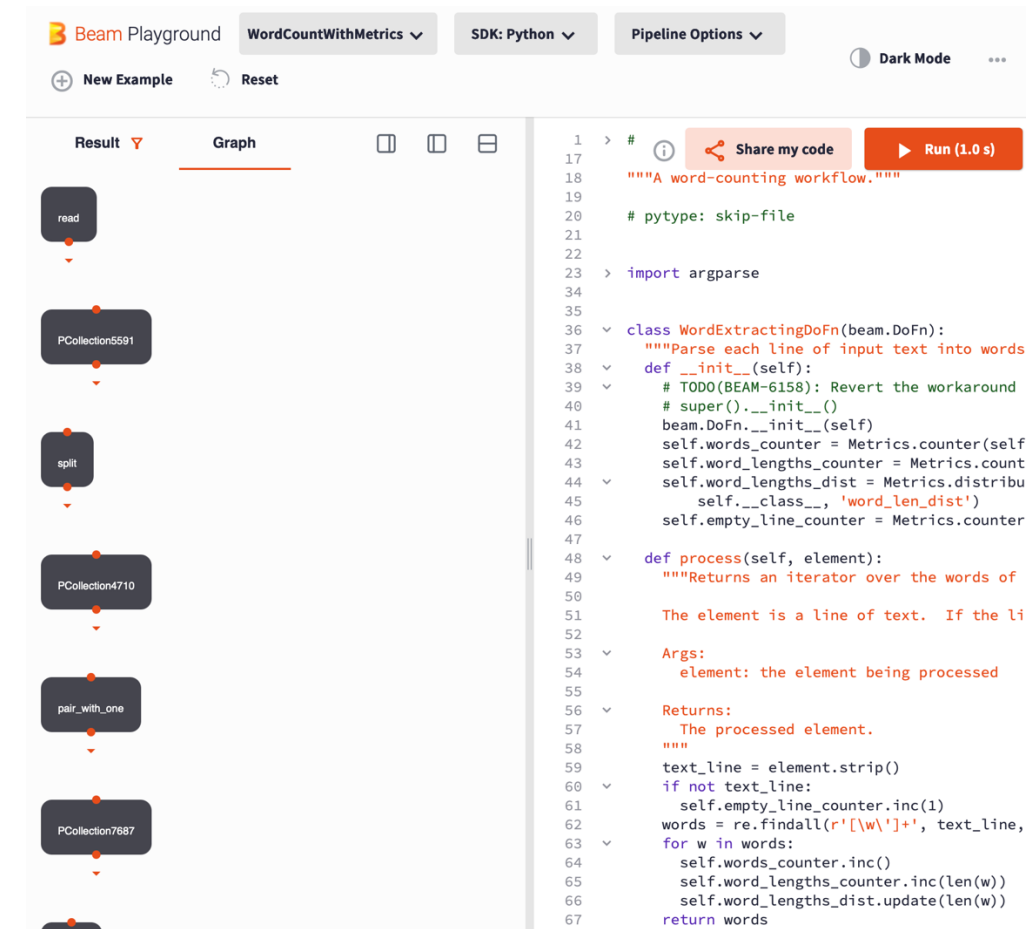
Apache Beam



<https://beam.apache.org/>

- It implements **batch and streaming data processing jobs** that run on any execution engine.
- A set of **SDK** frontends
- A set of Runners which can execute Beam jobs into various backends: Local, Apache Flink, Apache Spark, Apache Hadoop, Google Cloud Dataflow, ...
- If you are using **TFX**⁽¹⁾ for your pipeline tasks, you have already installed Apache Beam.
- Therefore, if you are looking for a minimal installation, reusing Beam to orchestrate is a logical choice.
- It is straightforward to set up, and it also allows you to use any existing distributed data processing infrastructure you might already be familiar with (e.g., **Google Cloud Dataflow**).
- You can also use Beam as an intermediate step to ensure your pipeline runs correctly before moving to Airflow or Kubeflow Pipelines.

(1) TensorFlow Extended



The screenshot displays the Apache Beam Playground interface. At the top, there are tabs for 'WordCountWithMetrics', 'SDK: Python', and 'Pipeline Options'. Below these are buttons for 'New Example' and 'Reset'. The main area is split into two panels: 'Result' and 'Graph'. The 'Graph' panel shows a pipeline with nodes: 'read', 'PCollection5591', 'split', 'PCollection4710', 'pair_with_one', and 'PCollection7687'. The 'Result' panel shows the output of the pipeline. On the right, there is a code editor with Python code for a word counting pipeline. The code includes a class 'WordExtractingDoFn' that implements the 'process' method to count words in a line of text. A 'Run (1.0 s)' button is visible next to the code.

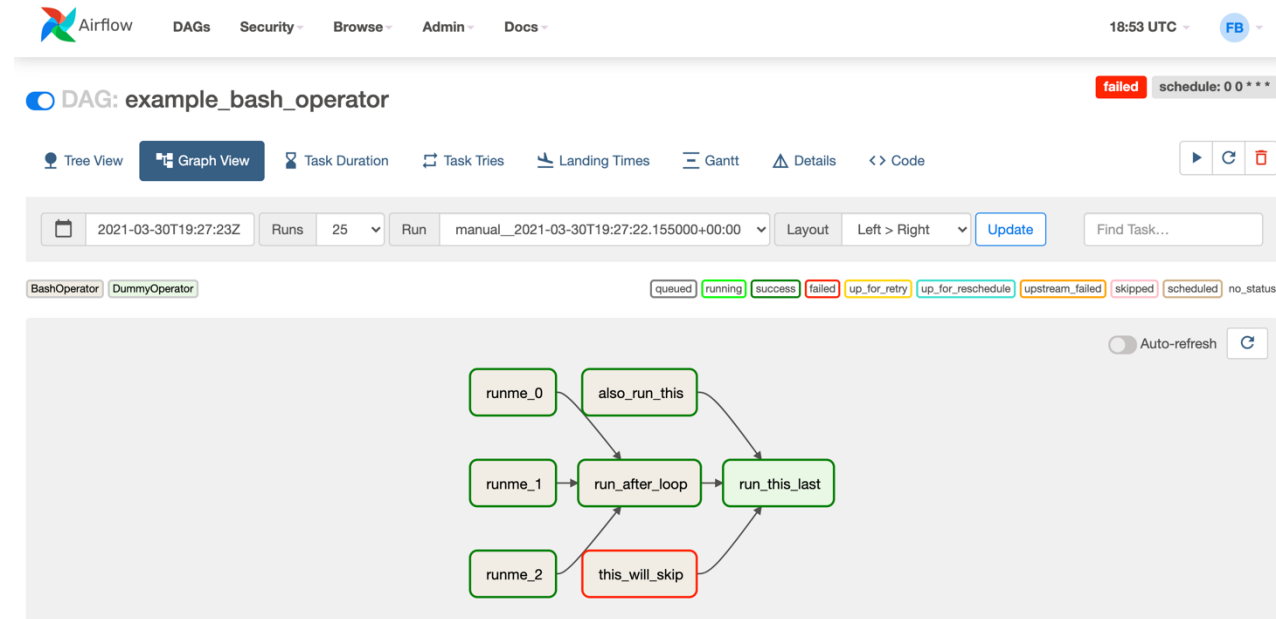
```
1 > #  Share my code  Run (1.0 s)
17
18 """A word-counting workflow."""
19
20 # pytype: skip-file
21
22
23 > import argparse
24
25
26 > class WordExtractingDoFn(beam.DoFn):
27     """Parse each line of input text into words
28     """
29     def __init__(self):
30         # TODO(BEAM-6158): Revert the workaround
31         # super().__init__()
32         beam.DoFn.__init__(self)
33         self.words_counter = Metrics.counter(self)
34         self.word_lengths_counter = Metrics.counter(self)
35         self.word_lengths_dist = Metrics.distribution(self)
36         self.__class__, 'word_len_dist')
37         self.empty_line_counter = Metrics.counter
38
39     def process(self, element):
40         """Returns an iterator over the words of
41         The element is a line of text. If the li
42
43         Args:
44             element: the element being processed
45
46         Returns:
47             The processed element.
48         """
49         text_line = element.strip()
50         if not text_line:
51             self.empty_line_counter.inc(1)
52         words = re.findall(r'[\w\']+', text_line)
53         for w in words:
54             self.words_counter.inc()
55             self.word_lengths_counter.inc(len(w))
56             self.word_lengths_dist.update(len(w))
57         return words
```

Apache Airflow



<https://airflow.apache.org/>

- Use Airflow to author workflows as directed acyclic graphs (DAGs) of tasks.
- The Airflow scheduler executes your tasks on an array of workers while following the specified dependencies.
- Rich command lines utilities makes performing complex surgeries on DAGs a snap.
- The rich user interface makes it easy to visualize pipelines running in production, monitor progress and troubleshoot issues when needed.
- If you use Apache Airflow in combination with a production-ready database like PostgreSQL, you can take advantage of executing partial pipelines.
- This can save a significant amount of time if a time-consuming pipeline fails and you want to avoid rerunning all the previous pipeline steps.



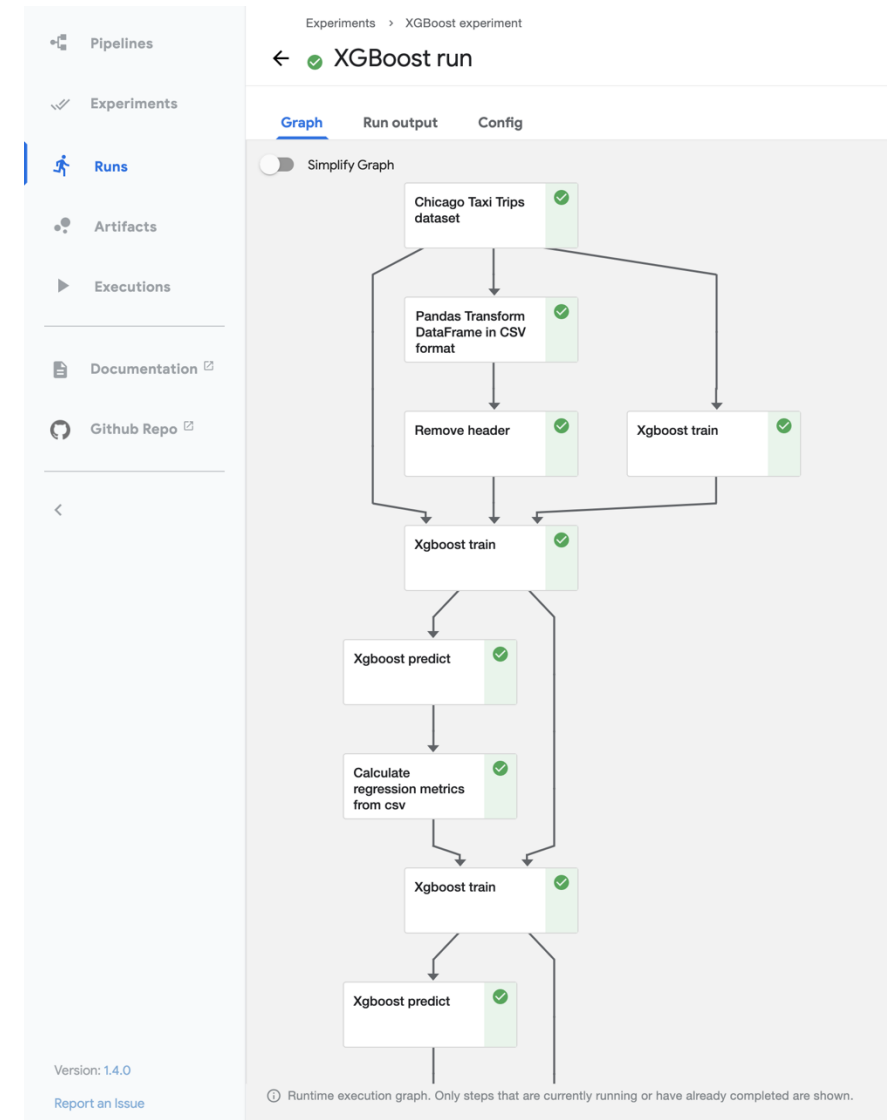
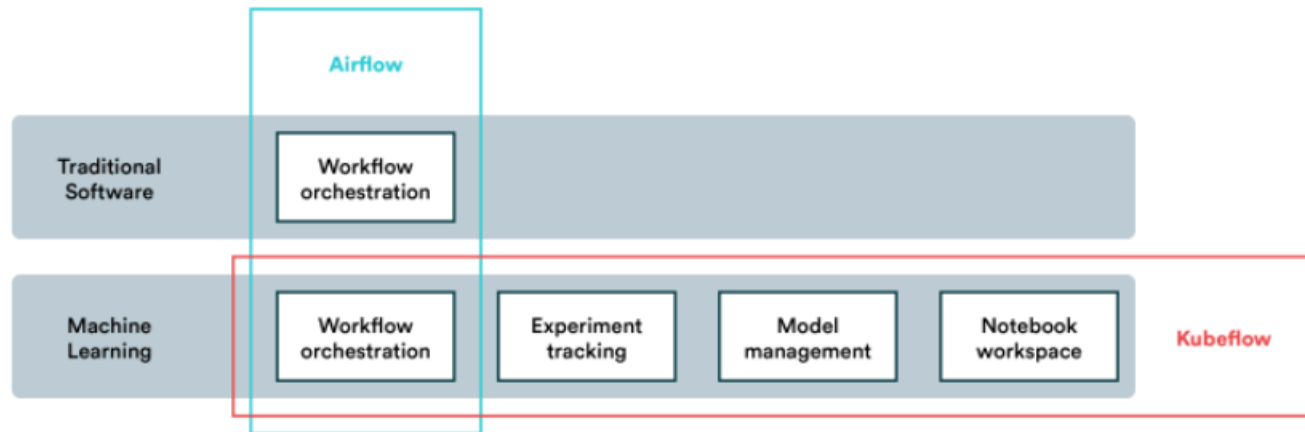
Kubeflow



<https://www.kubeflow.org/>

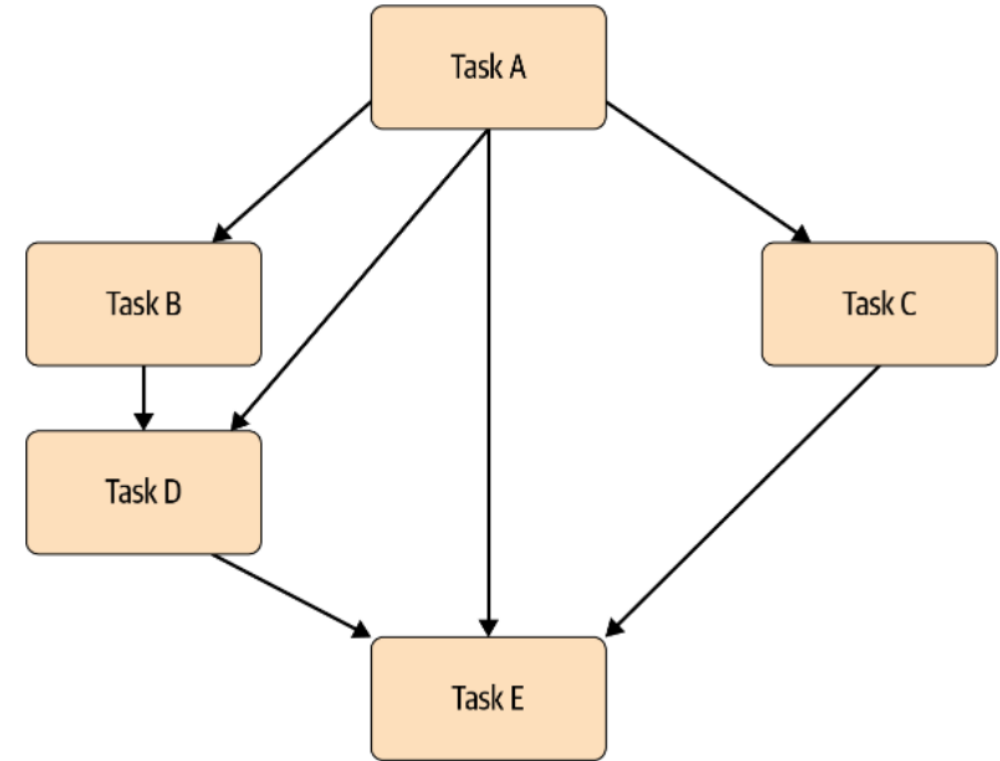
Kubeflow is a free and open-source ML platform that allows you to use ML pipelines to orchestrate complicated workflows running on Kubernetes. This solution was based on Google's method of deploying TensorFlow models, that is, TensorFlow Extended.

Kubeflow was created by Google to organize their internal machine learning exploration and productization, while Airflow was built by Airbnb to automate any software workflows.



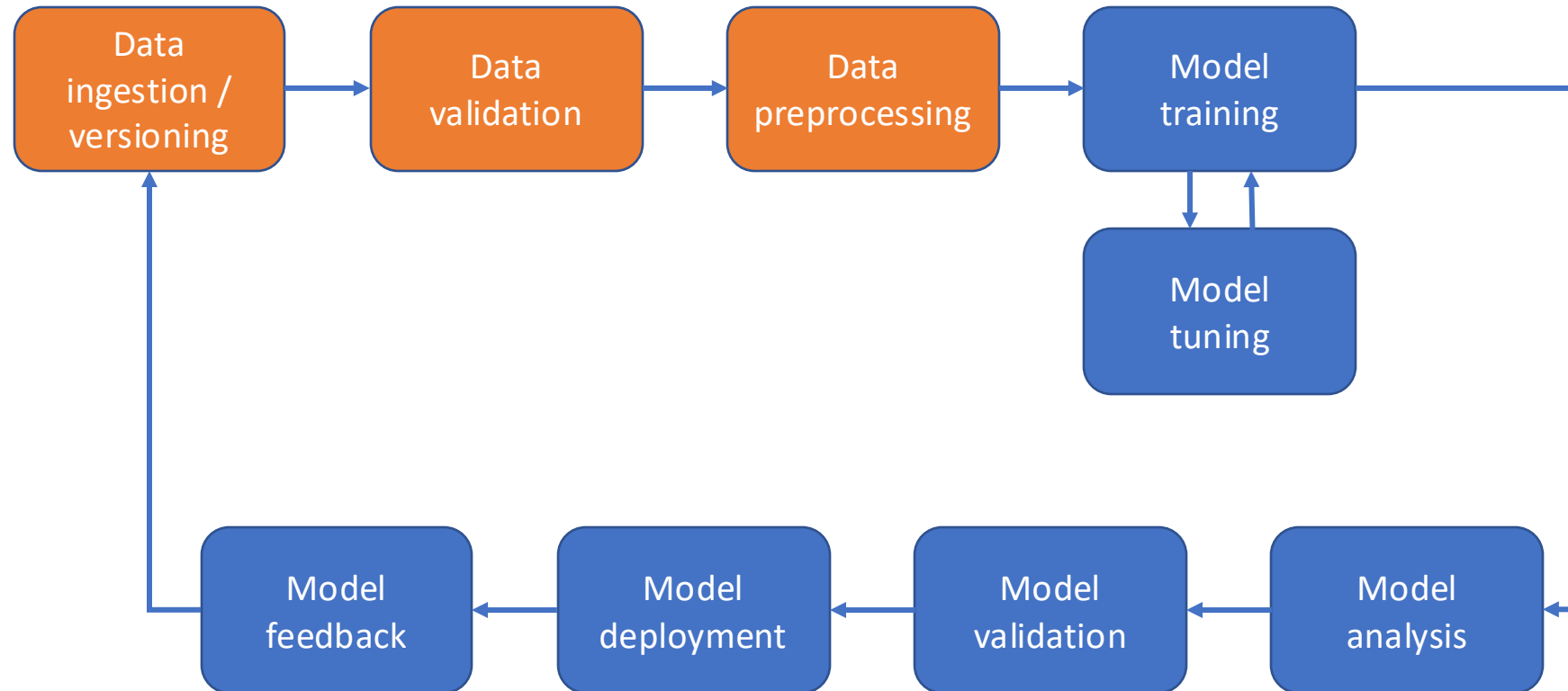
Directed Acyclic Graphs (DAG)

- Graph representation of task dependencies
- Directed graphs avoid situations where some tasks start without all dependencies fully computed
- Pipeline graphs must also be acyclic, meaning that a graph isn't linking to a previously completed task (a beginning and an end)
- DAGs are a central concept behind most workflow tools



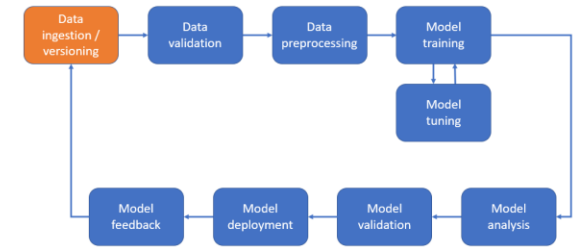
Data Pipelines

Data Pipelines

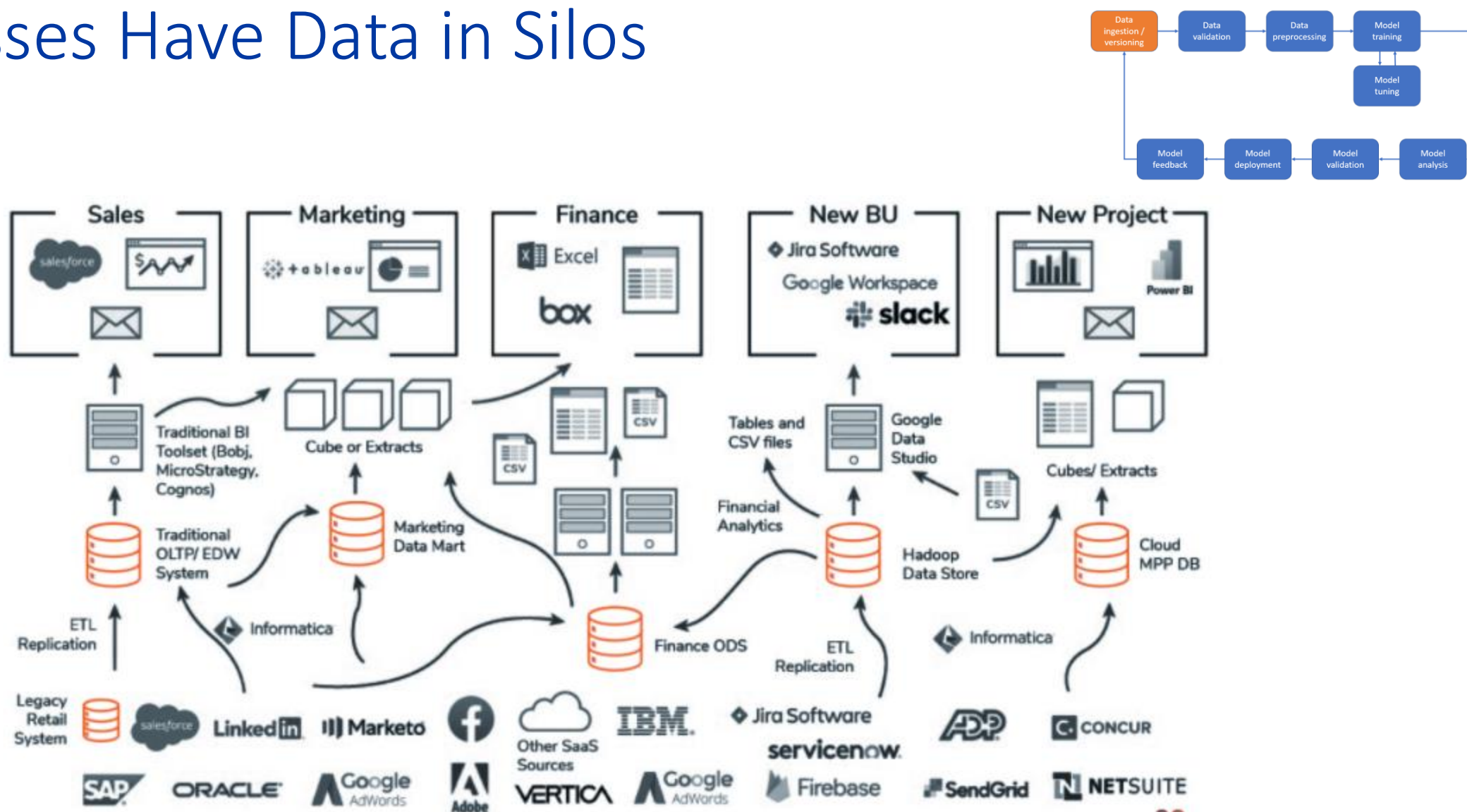


Key questions about data

- What relevant datasets are **available**?
- Is this data sufficiently accurate and **reliable**?
- How can stakeholders get **access** to this data?
- What data properties (known as features) can be made available by **combining multiple sources** of data?
- Will this data be available in **real time**?
- Is there a need to label some of the data with the “ground truth” that is to be predicted, or does unsupervised learning make sense? If so, how much will this **cost in terms of time and resources**?
- What **platform** should be used?
- How will data be **updated** once the model is deployed?
- How will the **KPIs**, which were established along with the business objectives, be measured?

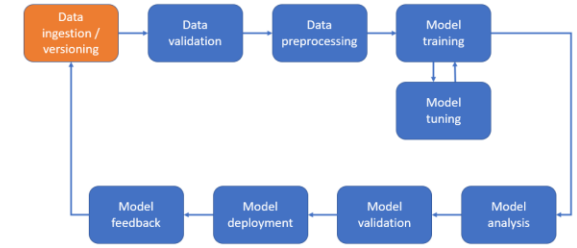


Businesses Have Data in Silos



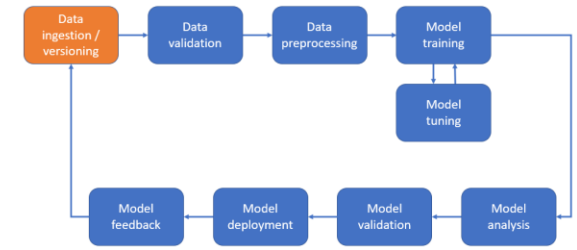
Data Some Basic Assertions

- Big amount of time to clean up and shape
- Data can come in different formats and different sources
- Better data quality -> Better model



Data Availability

- How close is our training data to the data that the model will see in real life?
- And how can we be sure that the models will be able to access similar data structures in both places?



Notice the difference!!!

Model Development

Model in Production

Model trained from data that is extracted from real-time systems and warehoused offline

Model run against real-time streaming data



Data structures must be the same

Model process a large amount of unstructured historical data (for example financial news)

Impossible to analyze unstructured data fast enough

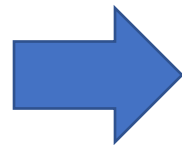
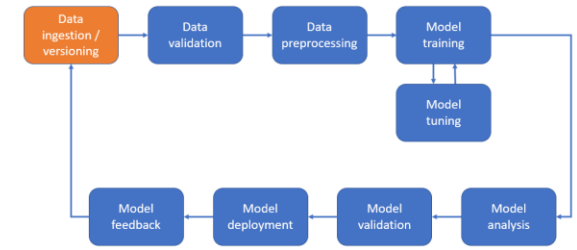


The data used in production must be obtainable



And data governance bring more questions

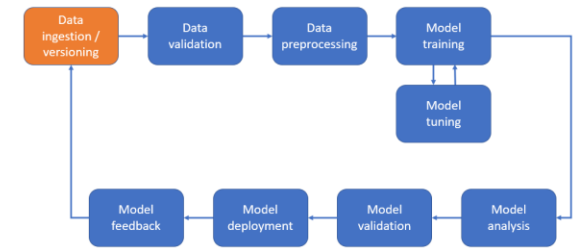
- Can the selected datasets be used for this purpose?
- Is there personally identifiable information (PII) that must be redacted or anonymized?
- Are there features, such as gender, that legally cannot be used in this business context?
- Are minority populations sufficiently well represented that the model has equivalent performances on each group?



GDPR (General Data Protection Regulation)
Bias

Data Ingestion

- Usually, a single step in the pipeline is responsible of **ingesting, splitting and converting** the datasets
- Datasets can be read from
 - Local data files
 - Remote data files (Google Cloud Storage or AWS Simple Storage Service S3)
 - Databases
- **Structured data**: often stored in a database or on a disk in file format, supporting tabular data
- **Text data for NLP**: this can grow to a considerable size. Use representation like Apache Parquet(1) to look for efficiency
- **Image data**: probably convert to from the image files to TFRecord(2) files, but not to decode the images
- Consider network traffic **costs** and **bottlenecks**



Notes:

(1) [Apache Parquet – Wikipedia](#)

(2) [TFRecords Explained – Towards Data Science](#)

Data Preparation

Splitting Datasets

Separate data into train/test
train/evaluation/test

Preserve existing splits

Spanning Datasets

Spans are like snapshots of data

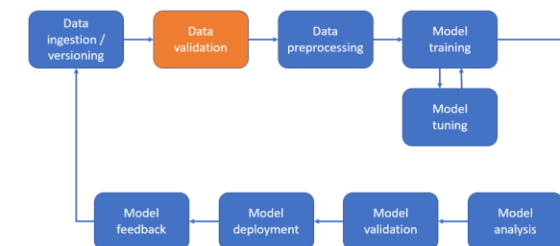
Every hour, day or week, a batch extract, transform, load (ETL) process could make such a data snapshot and create a new span.

Versioning Datasets

Track the produced models together with the used datasets

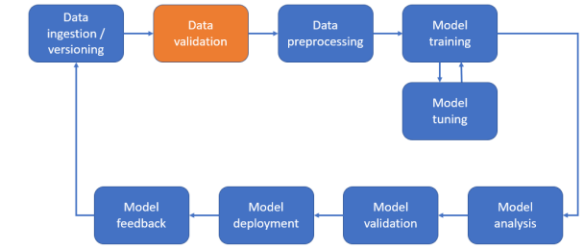
Not only store the file name and path of the ingested data, but a hash of the ingested data.

Such version tracking would allow us to verify that the dataset used during the training is still the dataset at a later point in time. Such a feature is critical for end-to-end ML reproducibility.



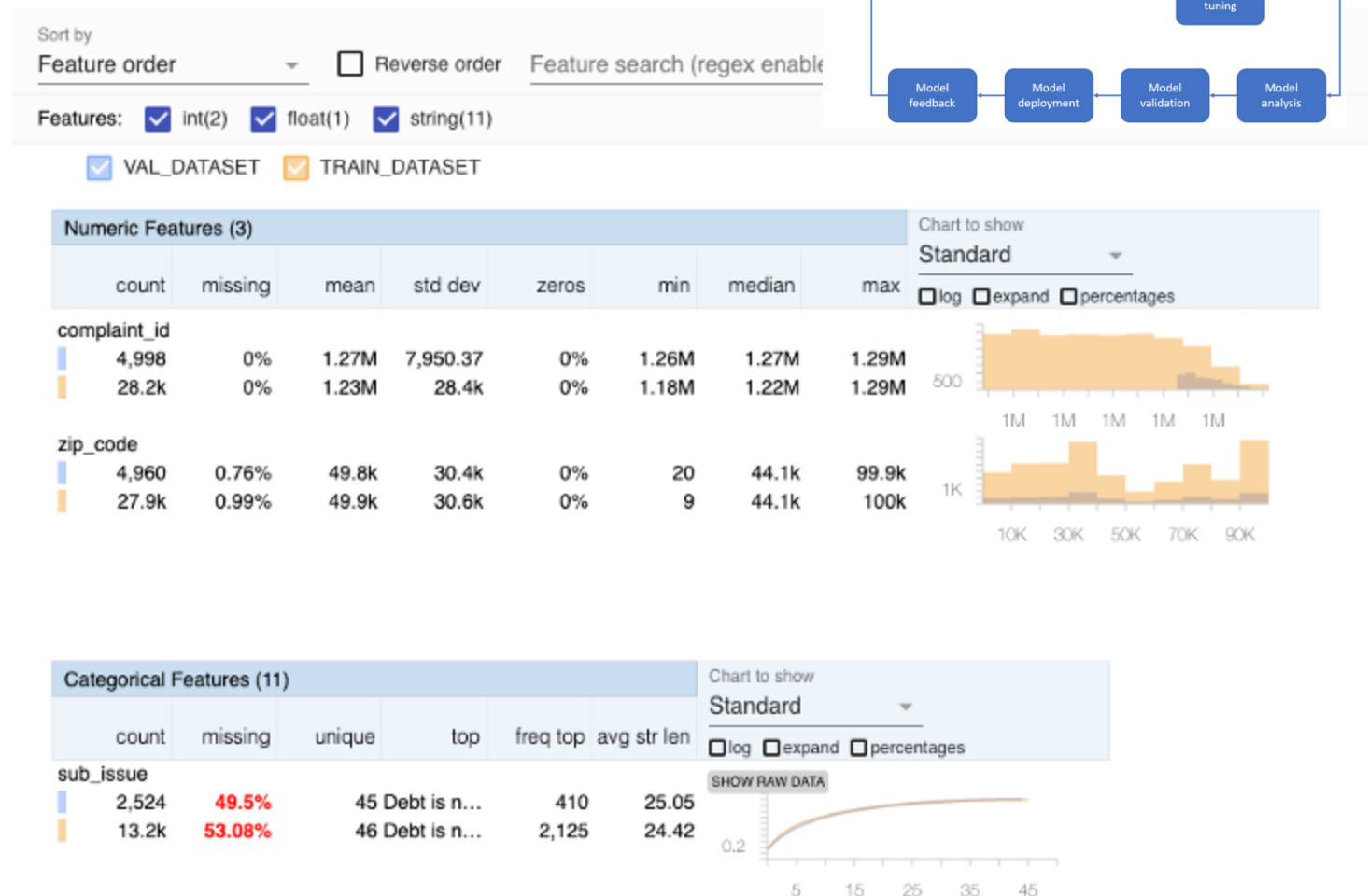
Why Data Validation?

- In machine learning, we are trying to learn from patterns in datasets and to generalize these learnings.
- This puts data front and center in our machine learning workflows, and the **quality of the data** becomes fundamental to the success of our machine learning projects.
- Every step in our machine learning pipeline determines whether the workflow can proceed to the next step or if the entire workflow needs to be abandoned and restarted (e.g., with more training data)
- Data validation is an especially important checkpoint because it catches changes in the data coming into the machine learning pipeline before it reaches the time-consuming preprocessing and training steps.



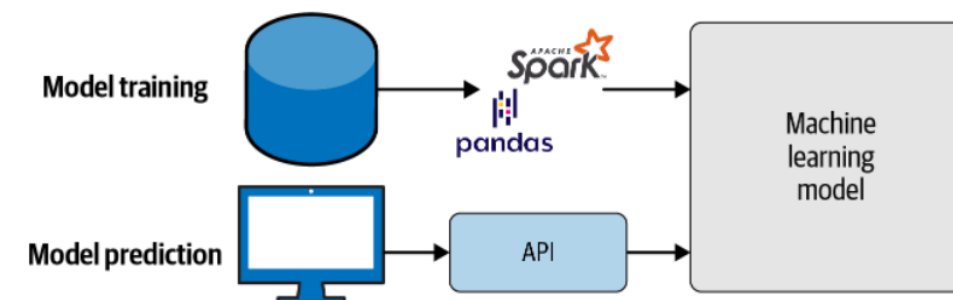
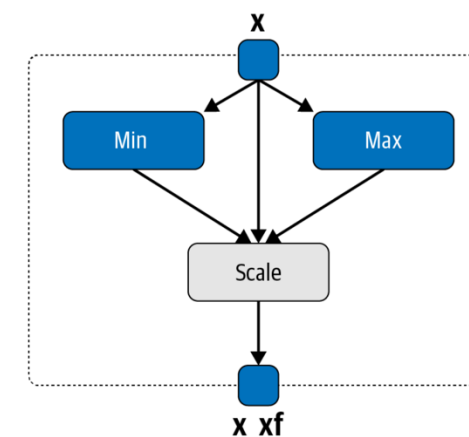
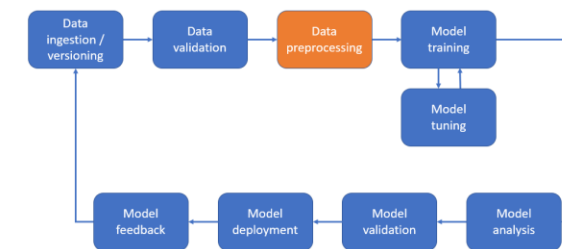
Data Validation

- If our goal is to automate our machine learning model updates, validating our data is essential.
- Three distinct checks on our data:
 - Check for data anomalies.
 - Check that the data schema hasn't changed.
 - Check that the statistics of our new datasets still align with statistics from our previous training datasets.
- Tools like TFDV can generate such checks



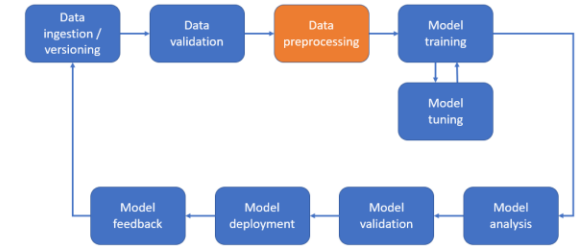
Data Preprocessing

- Why data preprocessing should be standardized in a machine learning pipeline:
 - Preprocessing your data efficiently in the context of the entire dataset
 - For example, if we want to normalize a numerical feature, we have to first determine the minimum and maximum values of the feature in the training set
 - Scaling the preprocessing steps effectively
 - Distribute preprocessing if needed
 - Avoiding a potential **training-serving skew**
 - When the preprocessing steps used during model training get out of line with the steps used during inference.
 - In many cases, the data used to train models is processed in Python notebooks with pandas or in Spark jobs.
 - When the model is deployed to a production setup, the preprocessing steps are implemented in an API before the data hits the model for the prediction

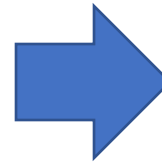


Feature Engineering

- How do we create new variables that give us more accurate models?
- Can the variables be replicated in an operational environment?



In feature engineering, data scientists analyze variables in the data to determine what transformations could be applied to these variables as well as what new variables could be created that would increase the predictive power of a machine learning model.



From a deployment point of view, it is critical that important features that have been engineered in this way **can be reproduced** in the operational system.

Remember:

Complexity

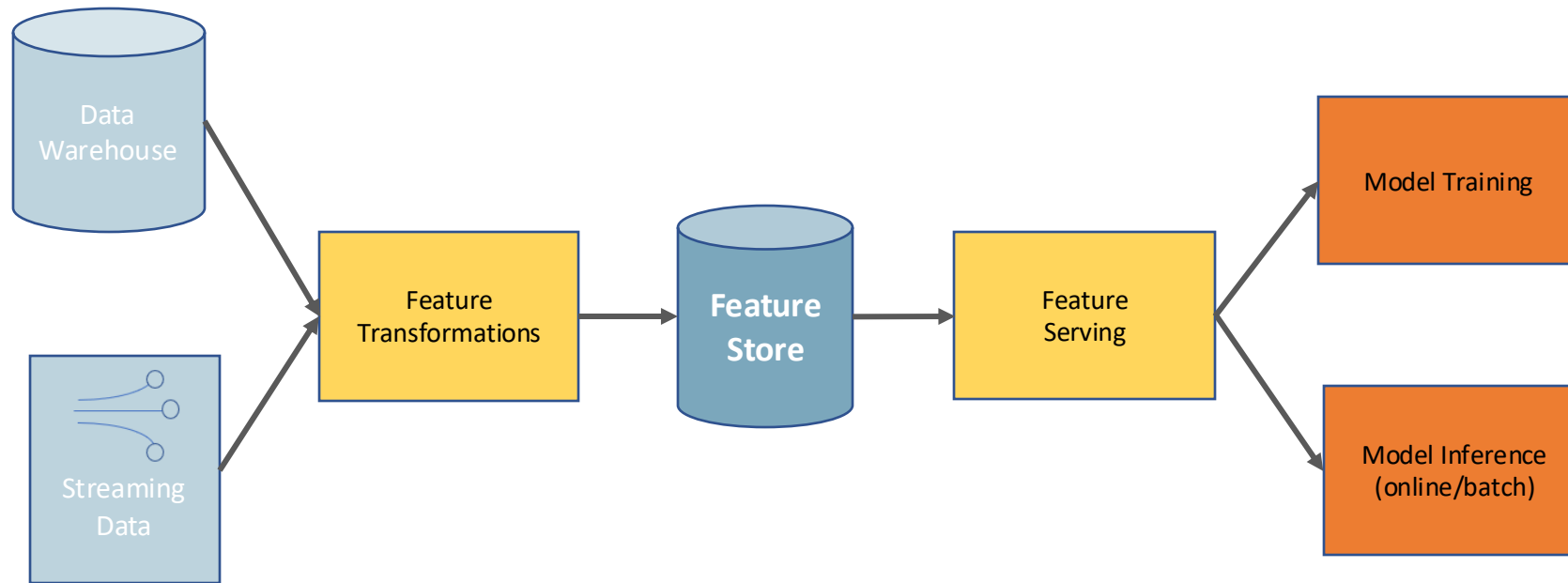
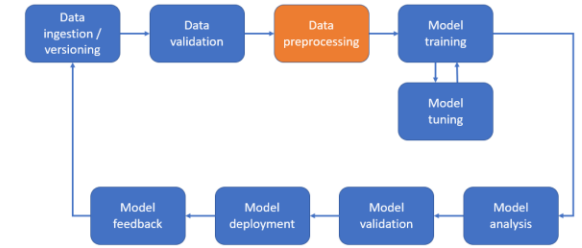
-> Greater Accuracy (probably)

-> Greater **Cost**

-> More **Difficult** to Explain

Feature Store

- A repository of frequently used features both for training/inference
- Data pipelines to convert raw data into features can be used by many models
- Facilitates feature reusability



Some concerns:

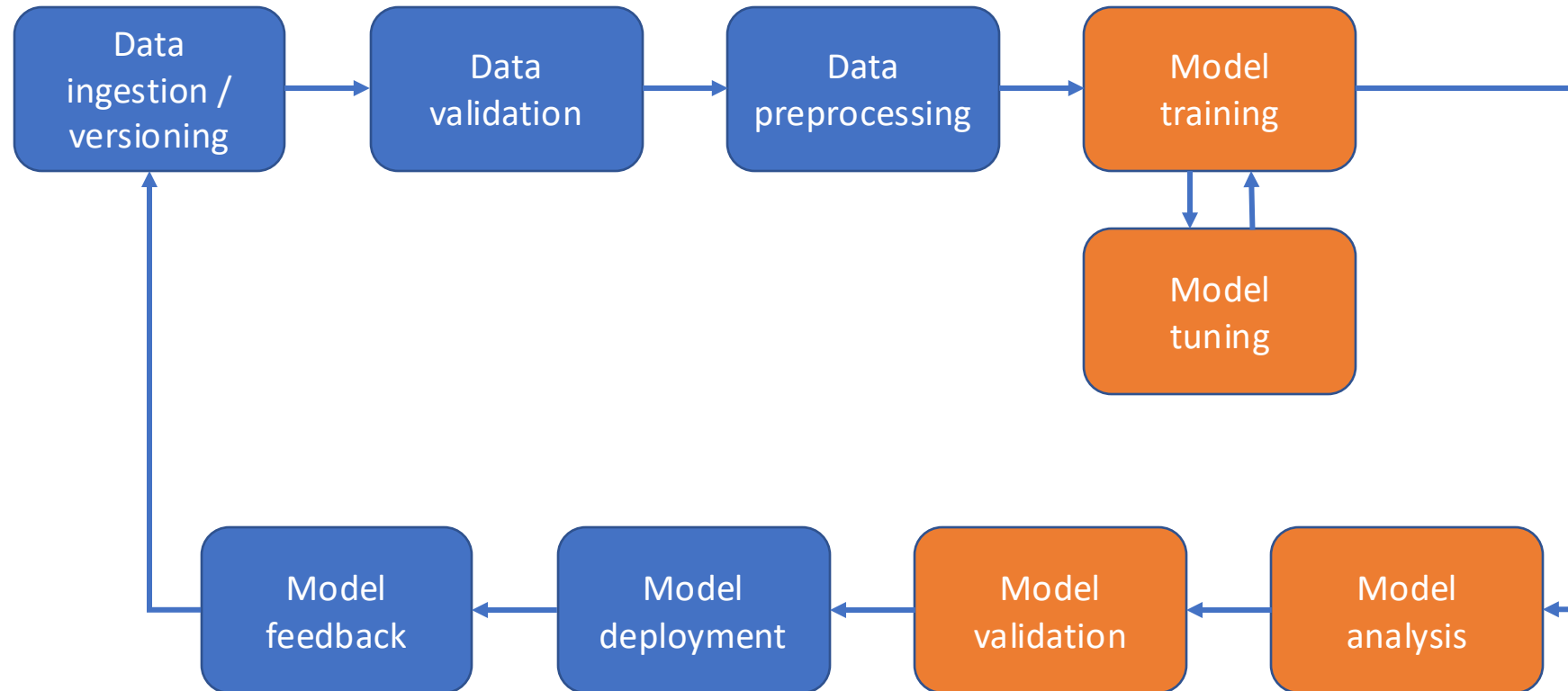
- Too rigid?
- Too complex?
- Model customization?

Apache Beam

Video: 'MLOps - 03P - Apache Beam-Flink'

Model Development Pipelines

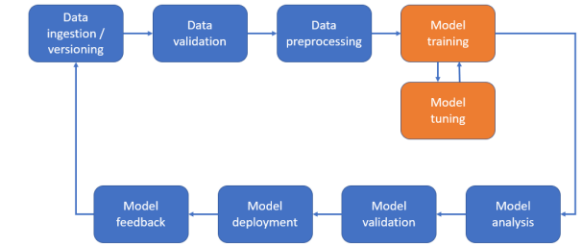
Model Development Pipelines



Model Training and Tuning

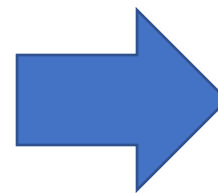
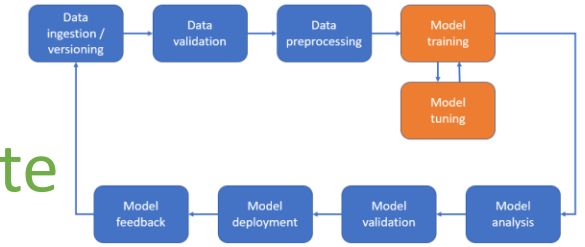
Experimentation goals:

- **Assessing how useful or how good** of a model can be built
- Finding the **best modeling parameters** (algorithms, hyperparameters, feature preprocessing, etc.)
- **Tuning the bias/variance trade-off** for a given training cost to fit that definition of “best.”
- Finding a balance between **model improvement and improved computation costs**. (Since there's always room for improvement, how good is good enough?)



Model Versioning

- When experimenting, data scientists need to be able to quickly **iterate** through all the possibilities for each of the model building blocks
- Trying all **combinations** of every possible hyperparameter, feature handling, etc., quickly becomes untraceable
- All or part of this process may need to be **repeated** every time anything in the situation changes
- Machine Learning platforms allow for **automating these workflows** not only on the first run, but also to preserve all the processing operations for repeatability



TFX
Mlflow
Kubeflow

...

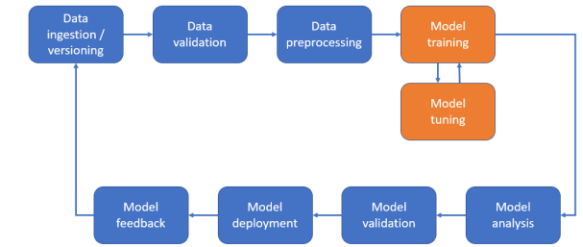
MLFlow



https://github.com/juanmaramirezgarcia/mlops_course_materials

Hyperparameter Tuning

- Hyperparameters are the variables that **govern the training process**
 - For example, part of setting up a deep neural network is deciding how many hidden layers of nodes to use between the input layer and the output layer, and how many nodes each layer should use. These variables are not directly related to the training data. **They are configuration variables**. They are usually constant during a job.
 - Hyperparameter tuning works by running **multiple trials** in a single training job. Each trial is a complete execution of your training application with values for your chosen hyperparameters, set within limits you specify.
 - There is very **little universal advice** to give about how to choose which hyperparameters you should tune. If you have experience with the machine learning technique that you're using, you may have insight into how its hyperparameters behave



AutoML

- What do we need/expect from AutoML?

Everything a good analyst would provide

Estimate predictive performance

Feature selection

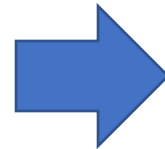
Explain and interpret the final model

Facilitate decision making

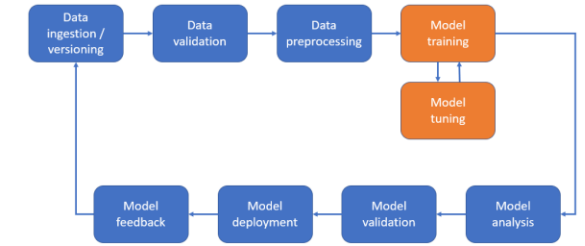
Create an operational environment for the model

Monitor execution – raise alarms

...

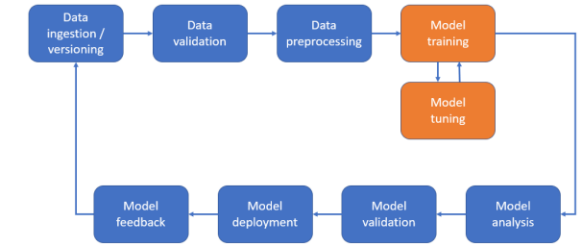


We are not there yet, but getting closer...



AutoML in the Cloud

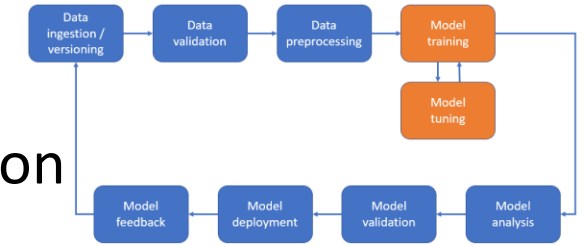
- AutoML in Azure Machine Learning
 - <https://docs.microsoft.com/en-us/azure/machine-learning/concept-automated-ml>
- Google Cloud AutoML
 - <https://cloud.google.com/automl>
- AWS AutoML Solutions
 - <https://aws.amazon.com/machine-learning/automl/>
- DataRobot Automated Machine Learning
 - <https://www.datarobot.com/platform/automated-machine-learning/>
- IBM AutoAI
 - <https://www.ibm.com/cloud/watson-studio/autoai>



CASH vs. Automated ML

CASH – Combined Algorithm Selection and Hyperparameter Optimization

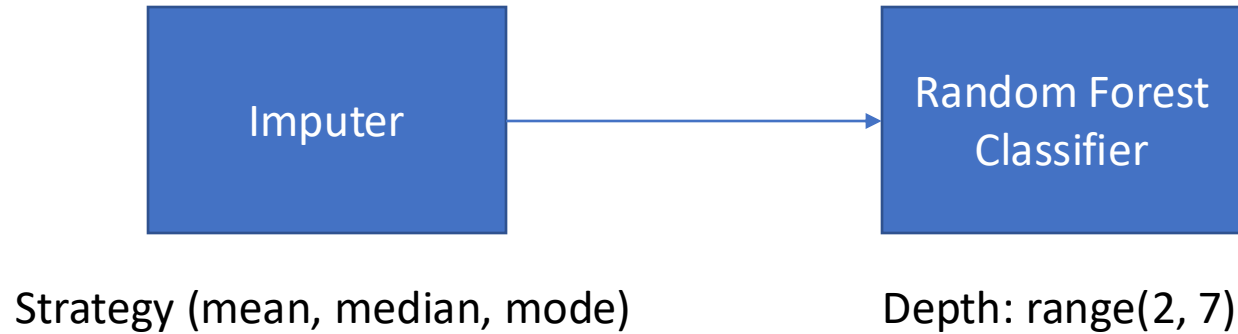
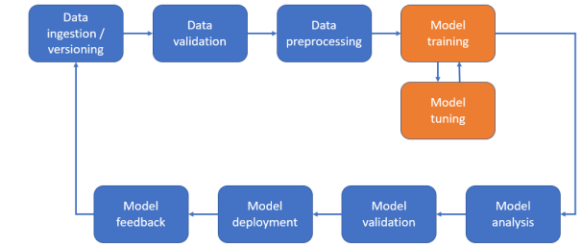
- Library of algorithms for each step in machine learning analysis:
 - Transformations
 - Imputations of missing values
 - Modeling
- Returns a predictive model



Example: **Auto-Sklearn**

Hyperparameter Optimization

- The objective of hyperparameter optimization is to find the optimal model pipeline components and their associated hyperparameters

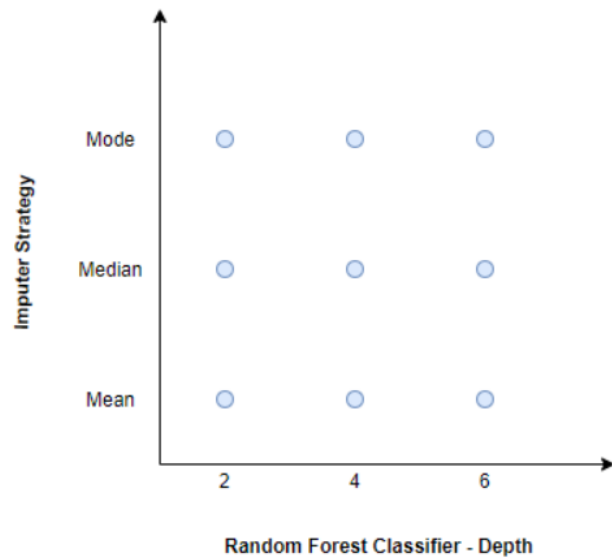


Two common ways: Grid Search or Random Search

Grid Search vs. Random Search

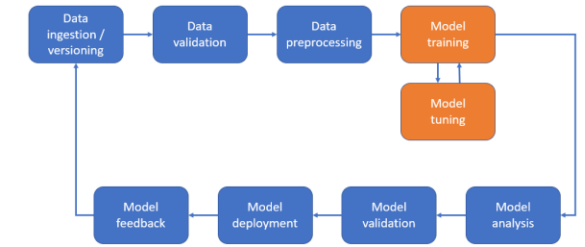
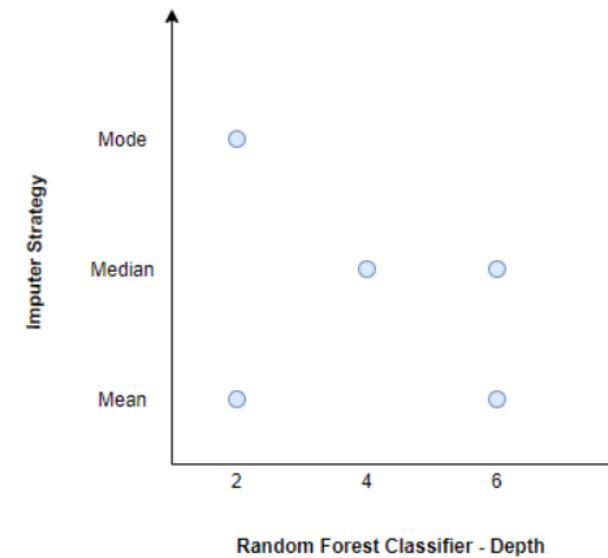
Grid Search

- For each hyperparameter, we make a list of possible values and try all possible combinations of values



Random Search

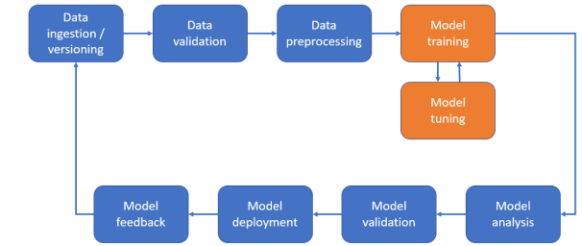
- We define the range and choices for each hyperparameter and the sets of hyperparameters are randomly chosen within these boundaries.



Notice that the sets of hyperparameters in Grid and Random Search are selected independently of one another

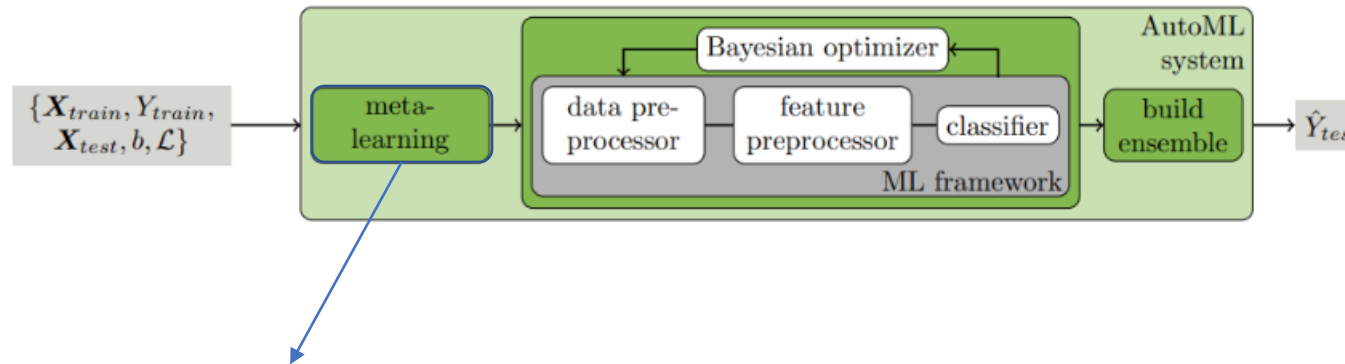
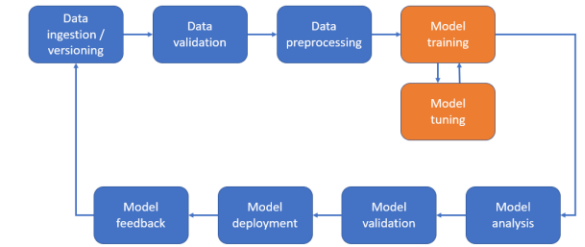
Bayesian Optimization

- A more efficient way to go about doing hyperparameter optimization is to utilize results from prior trials to improve the selection of hyperparameters for the next trial
- **Bayesian optimization** stores prior searched hyperparameters and results of a predefined objective function (e.g. binary cross entropy loss) and use it to create a **surrogate model**
- The purpose of a surrogate model is to quickly estimate the performance of the actual model given a particular set of candidate hyperparameter.
- As the number of trials increases, the surrogate model, updated with additional trial results, **improves and starts to recommend better candidate hyperparameters**.
- Suffers from 'cold start problem' as at the beginning there are no historical trials, so it starts with random search.



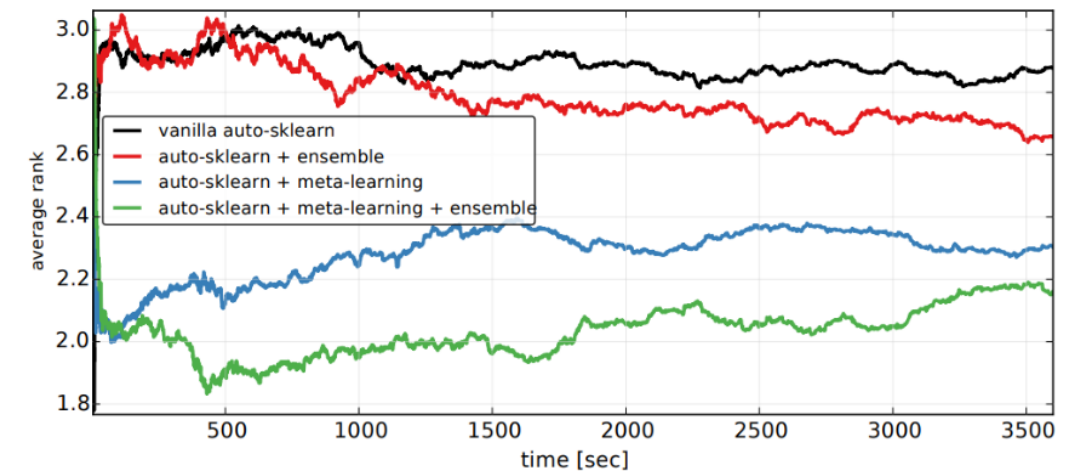
Auto-Sklearn <https://automl.github.io/auto-sklearn/master/#>

- Auto-Sklearn automates the tasks in the machine learning pipeline using the popular Scikit-Learn machine learning framework



The intuition behind meta learning is simple: **datasets with similar meta features performs similarly on the same set of hyperparameter.**

38 meta features such as skewness, kurtosis, number of features, number of classes, etc., were tabulated for 140 reference datasets from OpenML



Auto-Sklearn (cont.)

Data Pre-processors

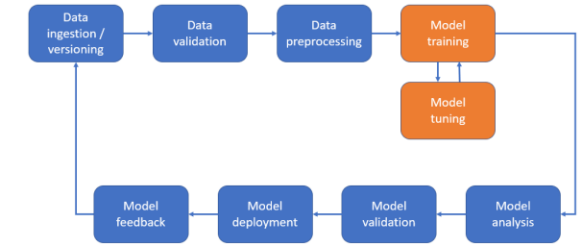
Auto-Sklearn preprocesses the data in the following order [2].

- One Hot Encoding of categorical features
- Imputation using mean, median or mode
- Rescaling features
- Balance the dataset using class weights

Feature Pre-processors

After data pre-processing, features may be optionally pre-processed with one or more of the following categories of feature pre-processors [2].

- Matrix decomposition using PCA, truncated SCV, kernel PCA or ICA
- Univariate feature selection
- Classification-based features selection
- Feature clustering
- Kernel approximations
- Polynomial feature expansion
- Feature embeddings
- Sparse representation and transformation



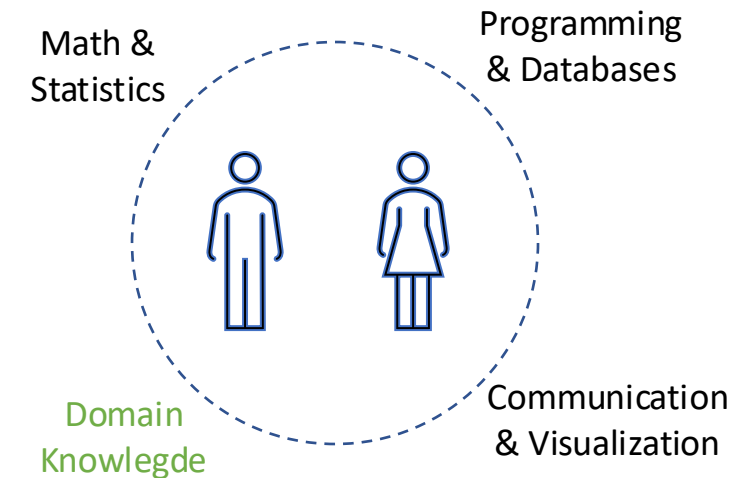
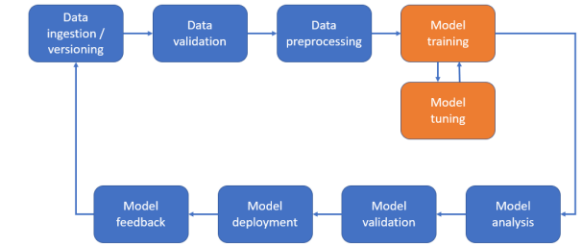
Ensemble

Ensemble models combines **weighted output of multiple trained models** to provide a final prediction.

They are known to be less prone to overfitting and generally outperforms single models.

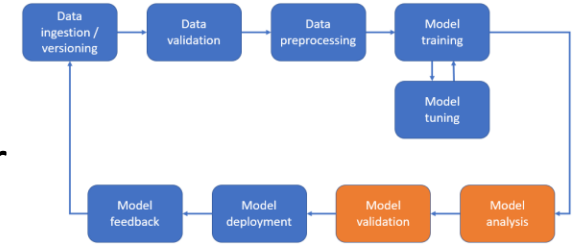
What is the future of AutoML

- The future of AutoML is bright
 - It is estimated that only 5% of total data is analyzed -> There is a big need of analysis
 - Manual scripting will be replaced by AutoML rather than script machine learning
 - Specialized AutoML will be imbedded in many applications
- This will not replace the Data Scientist: it will boost productivity and make the work of a data scientist more interesting and more challenging
- **Domain knowledge** will become more important



Model Analysis and Validation

- At this point in our machine learning pipeline, we have checked the statistics of our data, we have transformed our data into the correct features, and we have trained our model
- Still two extra steps before we move into deployment:
 - analyzing your model's performance in-depth
 - checking that it will be an improvement on any model that's already in production
- And careful: usually we use **a single metric** during all the training process!!!

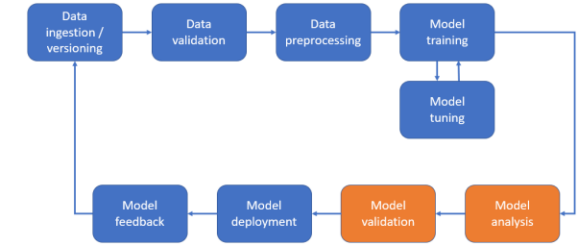


- It's essential to be able to monitor metrics across **different slices of your dataset**.
- It's also extremely important to **monitor your metrics through time**



Cross-Checking Model Behavior

Always challenge the model:



- **Cross-checking different metrics** (and not only the ones on which the model was initially optimized)
- **Checking how the model reacts to different inputs**—e.g., plot the average prediction (or probability for classification models) for different values of some inputs and see whether there are oddities or extreme variability
- **Splitting one particular dimension** and checking the difference in behavior and metrics across different subpopulations—e.g., is the error rate the same for males and females?

AutoML with AutoSklearn

https://github.com/juanmaramirezgarcia/mlops_course_materials